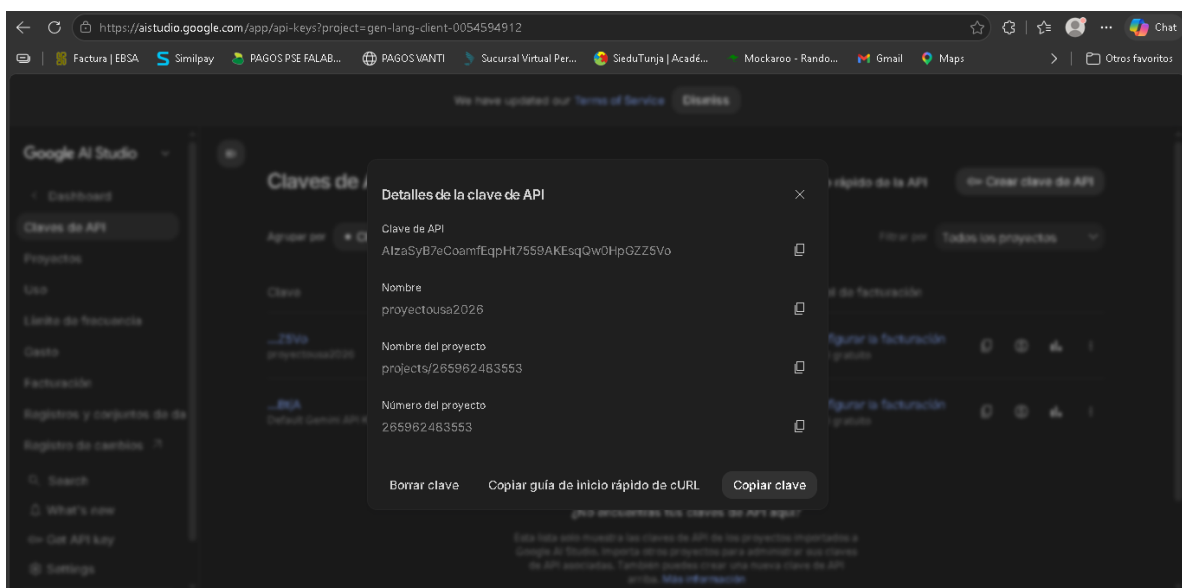
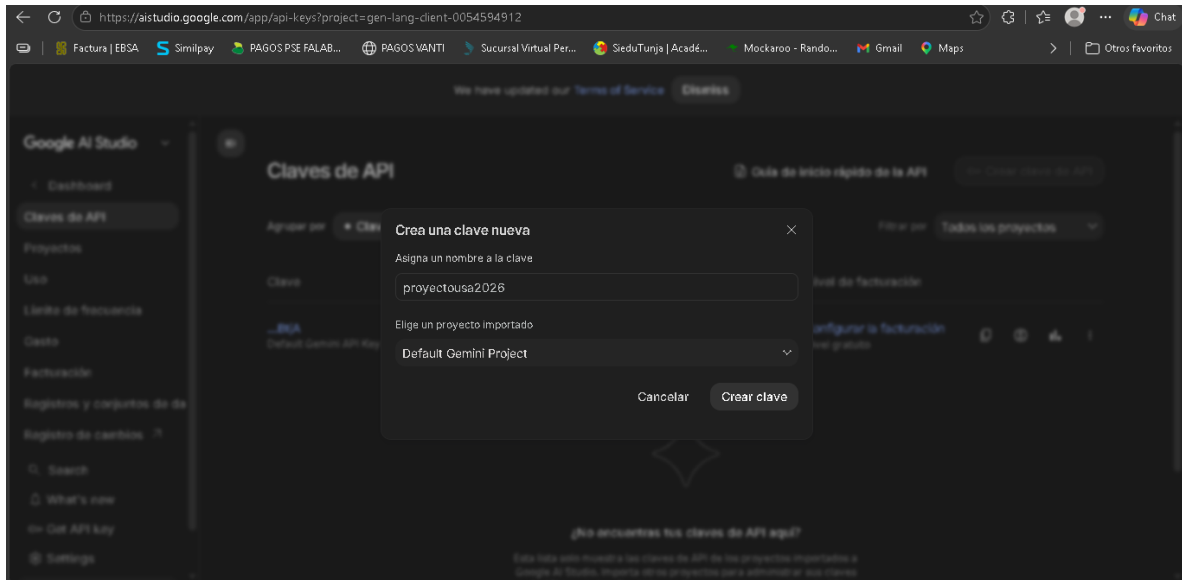
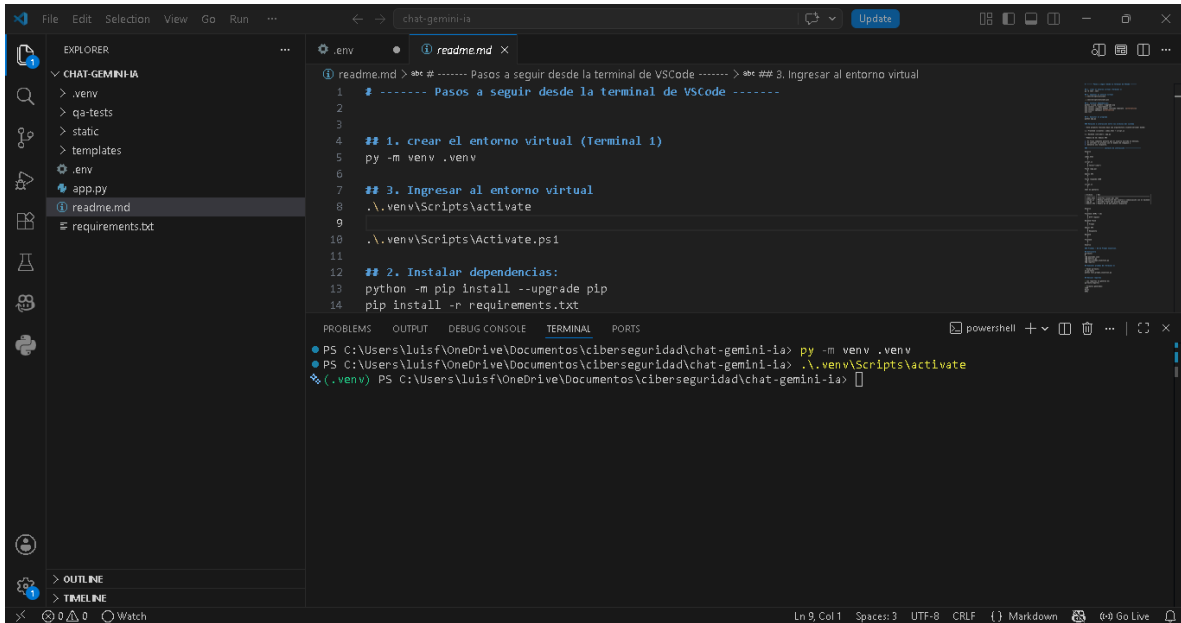


Creamos la API key que nos permitirá usar un modelo de IA de Gemini, el Gemini flash, después solo se debe copiar la clave y la pegamos en nuestro .env para comenzar las pruebas.



Creamos un entorno virtual donde vamos a instalar todas las dependencias necesarias para realizar las pruebas

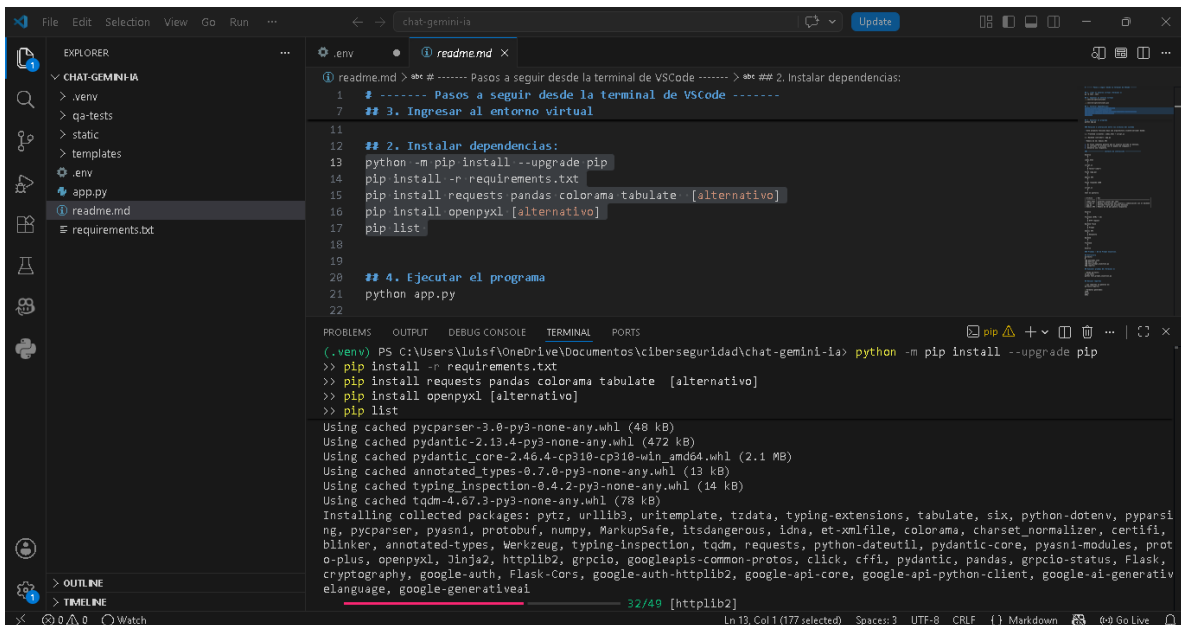


The screenshot shows the VS Code interface with the Explorer sidebar on the left showing the project structure for 'CHAT-GEMINI-IA'. The main editor displays a file named 'readme.md' with the following content:

```
1 # ----- Pasos a seguir desde la terminal de VSCode -----
2
3 ## 1. crear el entorno virtual (Terminal 1)
4 py -m venv .venv
5
6 ## 3. Ingresar al entorno virtual
7 .\venv\Scripts\activate
8
9 .\venv\Scripts\Activate.ps1
10
11 ## 2. Instalar dependencias:
12 python -m pip install --upgrade pip
13 pip install -r requirements.txt
```

The terminal at the bottom shows the execution of these commands in a PowerShell session:

```
PS C:\Users\luisf\OneDrive\Documentos\ciberseguridad\chat-gemini-ia> py -m venv .venv
PS C:\Users\luisf\OneDrive\Documentos\ciberseguridad\chat-gemini-ia> .\venv\Scripts\activate
(.venv) PS C:\Users\luisf\OneDrive\Documentos\ciberseguridad\chat-gemini-ia>
```



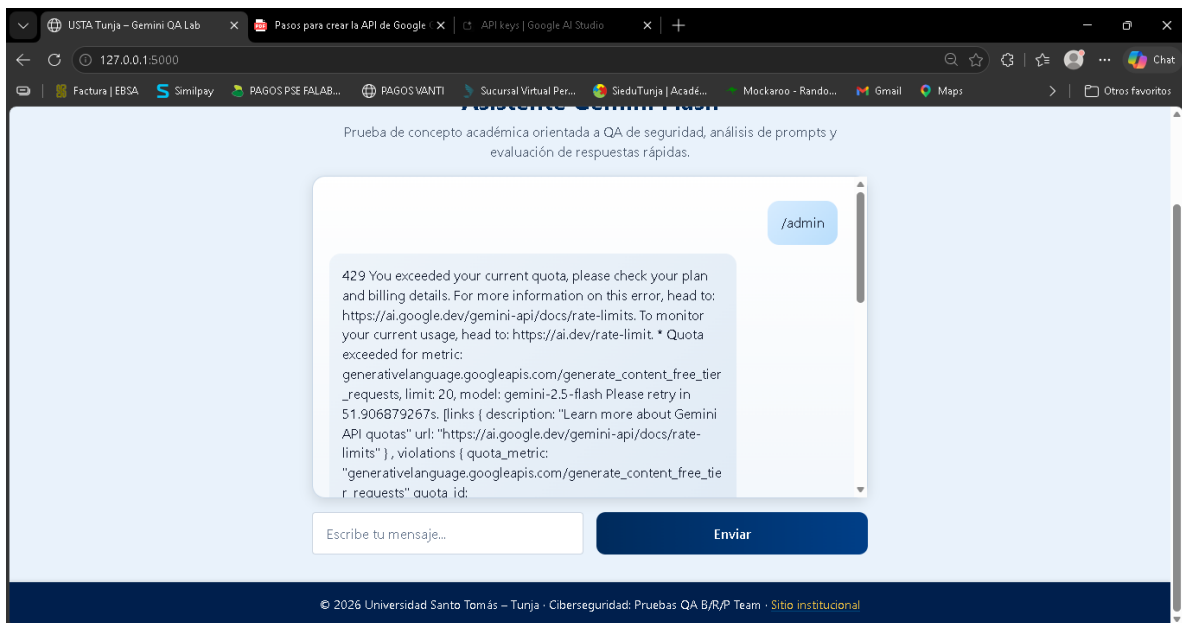
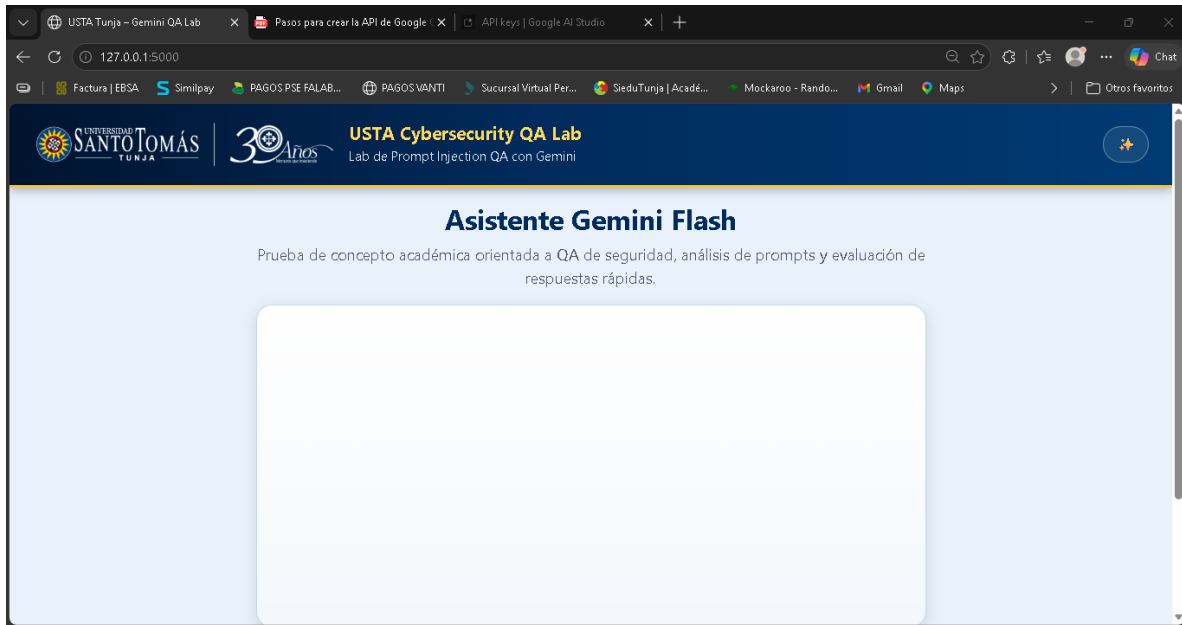
The screenshot shows the VS Code interface with the Explorer sidebar on the left showing the project structure for 'CHAT-GEMINI-IA'. The main editor displays a file named 'readme.md' with the following content:

```
1 # ----- Pasos a seguir desde la terminal de VSCode -----
2
3 ## 3. Ingresar al entorno virtual
4
5 ## 2. Instalar dependencias:
6 python -m pip install --upgrade pip
7 pip install -r requirements.txt
8 pip install requests pandas colorama tabulate [alternativo]
9 pip install openpyxl [alternativo]
10 pip list
11
12 ## 4. Ejecutar el programa
13 python app.py
```

The terminal at the bottom shows the execution of these commands in a PowerShell session:

```
(.venv) PS C:\Users\luisf\OneDrive\Documentos\ciberseguridad\chat-gemini-ia> python -m pip install --upgrade pip
>> pip install -r requirements.txt
>> pip install requests pandas colorama tabulate [alternativo]
>> pip install openpyxl [alternativo]
>> pip list
Using cached pycparser-3.0-py3-none-any.whl (40 kB)
Using cached pydantic-2.13.4-py3-none-any.whl (472 kB)
Using cached pydantic_core-2.46.4-cp310-cp310-win_amd64.whl (2.1 MB)
Using cached annotated_types-0.7.0-py3-none-any.whl (13 kB)
Using cached typing_inspection-0.4.2-py3-none-any.whl (14 kB)
Using cached tqdm-4.67.3-py3-none-any.whl (78 kB)
Installing collected packages: pytz, urllib3, uritemplate, tridata, typing-extensions, tabulate, six, python-dotenv, pypars
ng, pycparser, pyasn1, protobuf, numpy, MarkupSafe, itsdangerous, ldna, et-xmlfile, colorama, charset-normalizer, certifi
, blinken, annotated-types, Werkzeug, typing-inspection, tqdm, requests, python-dateutil, pydantic-core, pyasn1-modules, prot
o-plus, openpyxl, Jinja2, httplib2, grpcio, googleapis-common-protos, click, cffi, pydantic, pandas, grpcio-status, Flask,
cryptography, google-auth, Flask-Cors, google-auth-httpb2, google-api-core, google-api-python-client, google-ai-generativ
elanguage, google-generativeai
32/49 [httplib2]
```

Verificamos el funcionamiento tanto de backend como de frontend y realizamos un pequeño ataque de prompt injection usando un /admin para comprobar el Role Escalation de forma manual.



Generamos el informe usando nuestro entorno virtual con python para probar la seguridad que tiene este chatbot usado ya en un entorno en producción sobre todos las prompt injection del top 10 OWASP LLMs evidenciando que cuenta con las suficientes medidas para contrarrestar cada uno.

